Talkly Technical Architecture Document

1. System Overview

Talkly is a multi-tenant AI voice-calling platform for outbound campaigns and inbound call handling. Companies (tenants) configure AI voice agents (backed by Bolna), upload leads, run outbound calling campaigns, receive inbound calls routed through Twilio, and get post-call transcript analysis and billing based on call minutes.

The system is a FastAPI/Python backend with a MongoDB document store, and a Next.js/React frontend.

2. Technology Stack

Backend (`backend/requirements.txt`): - FastAPI + Uvicorn (ASGI server) - `motor==3.1.1` / `pymongo==4.3.3` — async MongoDB driver - `PyJWT` — JWT auth - `passlib[bcrypt]` + `bcrypt==3.2.2` — password hashing - `openai` — GPT-4o / Whisper API client - `pydub` — audio processing (requires `ffmpeg` on host) - `openpyxl` — Excel import/export (lead uploads, reports) - `python-dotenv==1.0.1` , `requests` , `python-multipart` - Dev/test only (`requirements-dev.txt`): `pytest>=8` , `pytest-asyncio` , `httpx`

Frontend (`frontend/package.json`): - Next.js `16.3.0` , React `19` , TypeScript `5.7.3` - Tailwind CSS `4.3.3` - `shadcn` CLI over **Base UI** (`@base-ui/react`) primitives — not Radix - `framer-motion` / `motion` , `@tsparticles/*` (landing page effects), `lucide-react` (icons), `next-themes` (dark mode) - `@vercel/analytics` — the only host-specific dependency (suggests Vercel as intended frontend host, though no `vercel.json` is committed) - No global state library (no Redux/Zustand/React Query) — state is local React state plus a single `AuthProvider` context

3. Repository Structure

```
backend/
 - server.py             # FastAPI app entrypoint; also hosts the Bolna call-trigger endpoints directly
 - auth.py               # JWT auth, roles, tenant-isolation helpers
 - mongodb.py            # Motor client/db accessor
 - process_audio.py      # Post-call transcript analysis (OpenAI/OpenRouter)
 - report_service.py
 - inbound/              # controller.py, routes.py, session_manager.py, webhook.py — Twilio inbound h
 - routes/               # auth_routes, billing_routes, admin_routes, campaign_routes, lead_routes, ag
 - services/             # billing_service, crm_service, followup_service, intelligence_service, lang
 - telephony/            # bolna_provider.py, provider_base.py — written but unused (server.py duplic
 - legacy_sqlite/        # dead code, pre-Mongo predecessor
 - tests/                # conftest.py, test_tenant_isolation.py
 - Dockerfile

frontend/
 - app/                  # Next.js App Router
     - admin/ (access-requests/, companies/[id]/, users/)
     - agent/, analytics/, billing/, calls/, campaigns/, dashboard/,
       leads/, login/, prompt/, request-access/, settings/, team/, voice-intelligence/
     - page.tsx (landing), layout.tsx
 - components/
     - auth/RouteGuard.tsx
     - landing/          # marketing page sections
     - layout/AppLayout.tsx # authenticated shell
     - providers/AuthProvider.tsx
     - ui/               # small custom set (Card, MetricCard, MiniChart, button, border-beam)
```

```
- lib/ (api.ts, utils.ts)
```

`backend/` also contains ~40 ad-hoc `scratch_*.py` / `check_*.py` debug scripts at the root — these are developer scratch tooling, not part of the application.

4. Backend Architecture

4.1 Routers registered in `server.py`

Router	Mount path
`auth_router`	`/api/v1/auth`
`billing_router`	`/api/v1/billing`
`admin_router`	`/api/v1/admin` and `/api/v1/super-admin` (same router, mounted twice — legacy alias)
`campaign_router`	`/api/v1`
`lead_router`	`/api/v1`
`agent_router`	`/api/v1`
`inbound_router`	`/api/v1/inbound`
`access_requests_router`	`/api/v1/access-requests` and `/api/v1/admin/access-requests` (mounted twice)

The outbound/bulk Bolna call-trigger endpoints are defined directly on `app` inside `server.py`, not in a `routes/` file.

4.2 API surface by file

- **auth_routes.py**: `POST /register`, `POST /login`, `POST /activate`, `GET /me`
- **billing_routes.py**: `GET /wallet`, `GET /transactions`, `GET /usage`, `POST /wallet/create-order`, `POST /wallet/verify-payment`, `POST /webhook/cashfree`
- **admin_routes.py**: dashboard/stats, full company CRUD + activate/deactivate/status, per-company overview/analytics/users/calls/campaigns/leads/billing/prompt, global user list/patch
- **campaign_routes.py**: `POST/GET /campaigns`, `PATCH/DELETE /campaigns/{id}`
- **lead_routes.py**: `POST /leads`, `POST /leads/upload` (CSV/Excel via openpyxl), `GET /leads`, bulk delete
- **agent_routes.py**: `GET/PUT /agent` (sets `bolna_agent_id` and per-language agent mappings — not model/voice config, see §9)
- **access_requests.py**: create/list/stats, approve/reject
- **inbound/routes.py**: number inventory + purchase, per-number webhook config, call webhook, call list/detail, transfer, hangup

4.3 Services

- `billing_service.py` — computes per-minute call cost and debits company wallets
- `language_service.py` — resolves which Bolna `agent_id` to use per company/language; detects mid-call language switches from transcripts
- `intelligence_service.py`, `crm_service.py`, `followup_service.py`, `transfer_service.py` — call-flow support logic
- `payment_provider.py` — `CashfreeProvider`, the sole payment integration

5. Database (MongoDB)

Accessed via `motor` (async). Collections actually referenced in code:

Collection	Purpose / key fields
`users`	`user_id`, `email`, `password_hash`, `role`, `company_id`, `is_acti` `PENDING_ACTIVATION`)
`companies`	`name`, `bolna_agent_id`, `status`, `draft_prompt` / `published_pr`
`calls`	Call records, inserted at trigger time and updated on Bolna webhook/status callback; `company_id`-stamped
`leads`	`name`, `phone`, `campaign_id`, `property`, `location`
`campaigns`	`name`, `property`; stats (`leads_count`, `calls_made`, `connected`, `interested`) are computed via aggregation on `leads`/`calls`, not stored directly
`transactions`	Payment/credit ledger: `company_id`, `payment_provider`, `provider_transaction_id`, `created_at`
`usage_records`	Per-call billing entries, source for usage reporting
`wallets`	One doc per `company_id`, balance updated via atomic increment
`company_subscriptions` / `subscription_plans`	Plan-based billing, coexists with the pay-per-minute wallet model
`prompts`	Per-company system prompt + welcome message, draft/published split
`language_mappings`	`company_id` + language -> Bolna `agent_id` + status

Collection	Purpose / key fields
`phone_numbers`	Inbound Twilio number inventory per company
`activation_tokens`	Email-activation tokens for admin-created users
`access_requests`	Self-serve signup/demo requests with approve/reject workflow

A `legacy_sqlite/` module (`db.py`, `models.py`) is dead code from a pre-Mongo version and is not wired into the running app.

6. Authentication & Multi-Tenancy

- **Scheme**: stateless JWT bearer, `HS256`, via `PyJWT` and `FastAPI.Security(HTTPBearer())`.
- [!] `JWT_SECRET_KEY` falls back to a hardcoded default (`"your-super-secret-key-change-this-in-production"`) if the env var is unset — must be verified set in every deployed environment.
- **Token lifetime**: 7 days (`ACCESS_TOKEN_EXPIRE_MINUTES = 60*24*7`).
- **Password hashing**: bcrypt via `passlib`.
- **Roles**: exactly two canonical roles, `super_admin` and `company_admin` (`normalize_role()` maps legacy casings to these). Unrecognized roles — including the separate hiring app's `ORGANIZATION_ADMIN`, which shares this database — pass through unchanged and simply fail Talkly's role gates; this is the isolation mechanism between the two apps.
- **Anti-privilege-escalation design**: the JWT only carries the user ID (`sub`). `get_current_user()` re-reads `role`, `company_id`, and `is_active` from `db.users` on every request, so a stale or tampered token cannot escalate role or switch tenant.
- **Tenant isolation helpers** (`auth.py`):
- `require_company_access` — rejects super-admins from tenant routes (must use `/api/v1/admin`) and rejects users without a `company_id`
- `tenant_filter()` — auto-injects `company_id` into Mongo queries unless caller is super-admin
- `enforce_no_foreign_company()` — 403s if a client payload supplies a different `company_id`
- `stamp_company()` — strips any client-supplied `company_id` and forces the caller's own
- `assert_owned()` — returns 404 (not 403) for cross-tenant document access, to avoid leaking existence

7. Frontend Architecture

- Next.js App Router; pages under `app/` cover the authenticated product surface (dashboard, calls, campaigns, leads, billing, analytics, voice-intelligence, settings, team, agent, prompt) plus a `admin/` area and a public `login` / `request-access` / landing page.
- `AuthProvider.tsx` (React Context, client component): on mount, reads a token from `localStorage` (via `lib/api.ts`) and calls `GET /api/v1/auth/me` to hydrate the user. Mirrors the backend's role normalization logic client-side. Exposes `user`, `loading`, `isSuperAdmin`, and `companyContext` — a super-admin-only

"acting as company X" context persisted to `localStorage` (`sa_company_context`). `logout()` does a hard redirect (`window.location.href`), not client-side routing.

- `RouteGuard.tsx` (`components/auth/`) gates authenticated routes.
- No dedicated design-system component library is generated — `components/ui/` holds a small hand-picked set, not the full shadcn output.

8. Third-Party Integrations

Service	Purpose	Notes
Bolna	Voice AI agent, outbound call execution	`BOLNA_API_KEY`. Talkly only sends `agent_id`, recipient number, webhook URL, and prompt-template variables — model/voice/STT/TTS/duration config for each agent lives entirely in Bolna's own dashboard, not in this codebase.
Twilio	Inbound call telephony leg	Inbound calls hit a Twilio webhook, then get forwarded via TwiML `<Dial><Sip>` to a Bolna SIP endpoint — an extra billed telephony leg on top of Bolna's own cost, for inbound only.
OpenAI	Post-call transcript analysis (`gpt-4o`, hardcoded) + Whisper STT	Primary path; billed per call.
OpenRouter	Fallback for transcript analysis (`gemini-2.0-flash-001`)	Only used if `OPENAI_API_KEY` is unset or the OpenAI call fails — a materially cheaper model that isn't the default.
Cashfree	Payment gateway for wallet top-ups	Sole payment integration. [!] `billing_routes.py` has a stale comment reading "Create Razorpay order" above the Cashfree call — there is no Razorpay code anywhere in the repo.
Gmail SMTP	Transactional email (activation links, access-request notifications)	Raw `smtplib` via `SMTP_HOST` / `PORT` / `USER` / `PASS` — no SendGrid/Mailgun/SES SDK.

No error-monitoring/APM tool (Sentry or similar) is integrated in either backend or frontend.

9. Billing & Payments

- Pay-per-minute model: `billing_service.py` hardcodes `DEFAULT_PRICING` — outbound: provider cost Rs. 8/min, customer price Rs. 12/min; inbound: provider cost Rs. 7/min, customer price Rs. 11/min. Duration is rounded up to the next full minute, minimum 1 minute billed. There is no per-component (STT/LLM/TTS/telecom) cost breakdown anywhere in code — that pricing table is a flat estimate, not reconciled against actual Bolna invoices.
- A parallel `company_subscriptions` / `subscription_plans` model exists alongside the wallet model.
- **Payment flow** (`billing_routes.py`): `POST /wallet/create-order` creates a Cashfree order tagged with `company_id`. Two independent paths then credit the wallet with the same `provider_transaction_id`: 1. `POST /wallet/verify-payment` — client-initiated poll; if Cashfree reports `order_status == "PAID"`, credits immediately. 2. `POST /webhook/cashfree` — server-to-server webhook, HMAC-SHA256 signature verified, credits on `PAYMENT_SUCCESS_WEBHOOK` + `SUCCESS` status, using `company_id` from the order's tags (not an authenticated session). Whether `add_credits` dedupes on transaction ID (to prevent a double credit from both paths firing) was not verified — worth confirming directly in `billing_service.py` before relying on it.
- [!] `payment_provider.py` falls back to **mock order data** (always `PAID`, webhook signature always valid) when `CASHFREE_APP_ID` / `CASHFREE_SECRET_KEY` are unset — billing "succeeds" automatically with no real payment in any environment missing those credentials. Should be loud/fail-closed rather than silently mocked in anything but local dev.

10. Deployment

- Only `backend/Dockerfile` exists in the repo (no frontend Dockerfile, no docker-compose, render.yaml, Procfile, fly.toml, or vercel.json).
- Backend image: `python:3.10-slim`, installs `ffmpeg` via apt (required by `pydub`), `pip install -r requirements.txt`, runs `uvicorn server:app --host 0.0.0.0 --port 8000`.
- Frontend has no committed hosting config; the `@vercel/analytics` dependency is the only signal it's intended for Vercel.
- No `.env.example` exists — required env vars (names recovered from `os.getenv` / `os.environ` calls, not from reading actual secrets): `MONGODB_URI`, `MONGODB_DB`, `JWT_SECRET_KEY`, `BOLNA_API_KEY`, `BASE_URL`, `OPENAI_API_KEY`, `OPENROUTER_API_KEY`, `OPENROUTER_MODEL`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `CASHFREE_APP_ID`, `CASHFREE_SECRET_KEY`, `CASHFREE_ENVIRONMENT`, `NEXT_PUBLIC_FRONTEND_URL`, `ALLOW_SELF_REGISTER`, `UNATTRIBUTED_INBOUND_COMPANY_ID`.

11. Known Issues / Technical Debt

- **Bolna voice-stack config isn't in this repo**: STT/LLM/TTS provider, model, and call-duration/timeout are configured entirely on Bolna's dashboard per `agent_id` — Talkly never sends `agent_config.transcriber/synthesizer/llm` or duration limits, only `agent_welcome_message`. The `ai_voice` / `voice_gender` / `regional_accent` fields collected in the UI are sent as `user_data` prompt variables, not as actual Bolna voice-selection parameters — they likely don't change the TTS voice at all.
- **Billing duration default**: if a Bolna webhook/poll omits `duration` or sends 0 (e.g. a call that failed instantly or never connected), `server.py` defaults it to 60 seconds before billing — silently padding billed minutes on high no-answer campaigns.
- **No call-duration cap enforced by Talkly** — runaway calls are only bounded by whatever's set on the Bolna agent itself.

- **`telephony/bolna_provider.py`** is a parallel, unused abstraction — the real call-trigger logic is duplicated inline in `server.py`.
- **Pricing table vs. reality**: `DEFAULT_PRICING` (Rs. 8/12 outbound, Rs. 7/11 inbound) is a hardcoded estimate never reconciled against actual Bolna invoices.
- **Mock payment fallback**: Cashfree payments silently auto-succeed when credentials are unset (§9).
- **Stale Razorpay comment** in `billing_routes.py` — no functional impact, but misleading to future readers.